🔷 1. What is Virtualization (Core Concept)

Virtualization is a technique in computing where physical hardware resources are abstracted into virtual (logical) resources. Instead of directly using physical CPU, memory, storage, and network devices, a software layer creates virtual versions of these components. This allows multiple independent computing environments to run on a single physical machine.

In simple terms, virtualization makes one physical computer behave like many computers simultaneously. Each of these virtual computers is called a Virtual Machine (VM). A VM contains its own operating system, applications, and virtual hardware, and it behaves exactly like a real machine, even though it is sharing physical resources with other VMs.

This abstraction is important because it separates the software environment from the underlying hardware, making systems more flexible and manageable .

🔷 2. Why Virtualization is Needed (Problems Without It)

Before virtualization, computing systems were tightly coupled with physical hardware. Each application typically required its own dedicated server. This approach created several serious problems.

One of the main issues was resource underutilization, where servers were not fully used, wasting CPU power, memory, and storage. Another problem was lack of flexibility, since resources could not be dynamically allocated based on demand. Systems were also highly dependent on specific hardware configurations, making it difficult to migrate applications between machines.

Additionally, managing multiple physical servers increased costs and complexity, including power consumption, cooling, and maintenance. Disaster recovery was also difficult, as restoring systems required manual effort and downtime. Applications were not portable and could not be easily moved between machines.

Virtualization solves all these issues by allowing efficient sharing, dynamic allocation, and easy management of resources.

🔷 3. Virtual Machine (VM) and Its Nature

A Virtual Machine is a software-based representation of a physical computer. It includes:

- Operating system
- Applications
- Virtual CPU, memory, storage, and network

Each VM runs independently and is isolated from others. Even though multiple VMs share the same hardware, each VM behaves as if it has its own dedicated resources. This illusion is created by the virtualization system.

A VM provides:

- Isolation → one VM does not affect another

- Encapsulation → entire system stored as files
- Portability → can move between machines

🔷 4. Role of Hypervisor (Very Important)

The most important component in virtualization is the Hypervisor, also called the Virtual Machine Monitor (VMM).

The hypervisor is a software layer that sits between the hardware and virtual machines. Its main role is to create, manage, and control virtual machines. It allocates physical resources such as CPU, memory, and storage to different VMs and ensures that they do not interfere with each other.

It also intercepts instructions from VMs and translates them into operations on physical hardware. Because of this, each VM believes it is directly interacting with hardware, even though it is not.

The hypervisor ensures:

- Resource allocation
- Isolation
- Security
- Efficient execution

🔷 5. Types of Hypervisors

Hypervisors are classified into two main types based on how they interact with hardware.

A Type 1 hypervisor, also known as a bare-metal hypervisor, runs directly on physical hardware without any operating system. It provides better performance and is commonly used in cloud environments.

A Type 2 hypervisor runs on top of an existing operating system. It behaves like a normal application and relies on the host OS for hardware interaction. It is easier to use but less efficient compared to Type 1.

🔷 6. Benefits of Virtualization

Virtualization provides multiple advantages that make it essential in modern systems.

It enables server consolidation, where multiple virtual machines run on a single physical server, reducing hardware usage. It also supports elasticity and scalability, allowing systems to dynamically adjust resources based on demand.

Virtualization improves reliability, as features like snapshots and cloning allow easy backup and recovery. It ensures isolation, meaning failures in one VM do not affect others. It also reduces energy consumption and operational costs by minimizing the number of physical servers.

🔷 7. Virtualization in Infrastructure

Virtualization converts physical resources into virtual equivalents. For example, a physical CPU is divided into multiple virtual CPUs (vCPUs), and physical memory is divided into virtual memory assigned to different VMs.

Similarly, storage is converted into virtual disks, and networking components like switches and routers are virtualized into software-based versions. This allows flexible and efficient management of computing resources.

🔷 8. Virtualization Reference Model

A virtualized system is composed of three key components.

The host is the physical machine that provides hardware resources. The virtualization layer (hypervisor) manages these resources and creates virtual environments. The guest refers to the virtual machine that runs on top of the virtualization layer.

The guest does not directly interact with hardware. Instead, all interactions are handled through the virtualization layer.

🔷 9. Interaction with Hardware (System Calls, Interrupts, Traps)

Applications do not directly communicate with hardware. Instead, they use the operating system as an intermediary.

When an application needs to perform an operation like reading a file or accessing a device, it uses a system call. The operating system then executes the request.

Hardware can also generate interrupts to notify the system about events, such as input from a keyboard. A trap is generated when an error or exception occurs, such as division by zero.

In a virtualized system, these interactions are controlled by the hypervisor to ensure proper execution.

🔷 10. Virtualization Techniques

Different techniques are used to implement virtualization.

In full virtualization, the hypervisor completely emulates hardware, allowing unmodified operating systems to run. In hardware-assisted virtualization, special CPU features (like Intel VT) improve performance by supporting virtualization directly.

In para-virtualization, the guest operating system is modified to communicate directly with the hypervisor using special calls. In partial virtualization, only some hardware components are virtualized.

🔷 11. Virtual Machine Lifecycle

A virtual machine goes through different states during its operation. It can be created, started, paused, resumed, or stopped. It can also be saved and restored. These states allow flexible management of virtual machines.

🔷 12. OS-Level Virtualization (Introduction to Containers)

OS-level virtualization is different from hardware virtualization. Instead of creating full virtual machines, it creates isolated environments within a single operating system.

All instances share the same OS kernel but operate independently. This approach is lightweight and efficient because it avoids the overhead of running multiple operating systems.

This concept forms the foundation for containers.

🔷 13. Containers (Modern Virtualization)

Containers are a form of virtualization that operate at the operating system level. A container packages an application along with its dependencies, allowing it to run consistently across different environments.

Unlike virtual machines, containers do not include a full operating system. Instead, they share the host OS kernel, making them lightweight and fast.

🔷 14. Containers vs Virtual Machines

Virtual machines virtualize hardware and include a full operating system, making them heavier and slower to start. Containers, on the other hand, virtualize at the OS level and share the host kernel, making them lightweight and faster.

VMs provide stronger isolation, while containers provide better performance and scalability.

🔷 15. Docker (Important Concept)

Docker is a popular platform used to implement containerization. It allows developers to build, package, and run applications in containers.

Docker uses components such as:

- Docker client
- Docker daemon
- Images
- Containers
- Registry

It simplifies application deployment and ensures consistency across environments.

🔷 16. Container Internals

Containers achieve isolation using techniques like namespaces, which separate processes, and control groups (cgroups), which manage resource allocation. They also use virtual networking and shared storage mechanisms.